

Cấu hình Spring Data JPA trong dự án Spring thuần (không Spring Boot)

Khi làm việc với Spring Data JPA trong một dự án Spring truyền thống (không sử dụng Spring Boot), bạn cần cấu hình thủ công một số thành phần quan trọng. Quá trình này bao gồm việc định nghĩa `DataSource`, `EntityManagerFactory`, `PlatformTransactionManager`, và kích hoạt Spring Data JPA repositories. Có hai cách chính để thực hiện cấu hình này: sử dụng JavaConfig (cấu hình dựa trên Java) hoặc XML Configuration (cấu hình dựa trên XML).

1. Cấu hình bằng JavaConfig

JavaConfig là phương pháp cấu hình hiện đại và được khuyến nghị trong các dự án Spring. Nó cho phép bạn định nghĩa các bean và cấu hình ứng dụng bằng cách sử dụng các lớp Java và annotations.

1.1. Dependencies cần thiết

Đảm bảo rằng bạn đã thêm các dependency sau vào file `pom.xml` của dự án:

XML

```
<dependencies>
  <!-- Spring Context -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.3.27</version> <!-- Sử dụng phiên bản Spring phù hợp -->
  </dependency>
  <!-- Spring ORM (cho JPA) -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-orm</artifactId>
    <version>5.3.27</version>
  </dependency>
  <!-- Spring Data JPA -->
  <dependency>
    <groupId>org.springframework.data</groupId>
    <artifactId>spring-data-jpa</artifactId>
    <version>2.7.17</version> <!-- Sử dụng phiên bản Spring Data JPA phù
```

```

hợp -->
</dependency>
<!-- Hibernate (hoặc JPA Provider khác) -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>5.6.15.Final</version> <!-- Sử dụng phiên bản Hibernate phù
hợp -->
</dependency>
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-entitymanager</artifactId>
    <version>5.6.15.Final</version>
</dependency>
<!-- Database Driver (ví dụ: H2 Database) -->
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <version>2.1.214</version>
</dependency>
<!-- Logging (ví dụ: Logback) -->
<dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
    <version>1.2.11</version>
</dependency>
</dependencies>

```

1.2. Lớp cấu hình Java

Tạo một lớp cấu hình (ví dụ: `JpaConfig.java`) và đánh dấu nó bằng `@Configuration` và `@EnableJpaRepositories` .

Java

```

package com.example.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.orm.jpa.JpaTransactionManager;
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
import org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter;
import org.springframework.transaction.PlatformTransactionManager;
import

```

```

org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.persistence.EntityManagerFactory;
import javax.sql.DataSource;
import java.util.Properties;

@Configuration
@EnableJpaRepositories(basePackages = "com.example.repository") // Chỉ định
package chứa các Repository interfaces.
@EnableTransactionManagement // Kích hoạt quản lý transaction bằng
annotation.
public class JpaConfig {

    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
        dataSource.setDriverClassName("org.h2.Driver");
        dataSource.setUrl("jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1"); // Sử dụng
H2 in-memory database.
        dataSource.setUsername("sa");
        dataSource.setPassword("");
        return dataSource;
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory() {
        LocalContainerEntityManagerFactoryBean em = new
LocalContainerEntityManagerFactoryBean();
        em.setDataSource(dataSource());
        em.setPackagesToScan("com.example.model"); // Chỉ định package chứa
các Entity classes.

        HibernateJpaVendorAdapter vendorAdapter = new
HibernateJpaVendorAdapter();
        vendorAdapter.setGenerateDdl(true); // Tự động tạo DDL (Data
Definition Language) từ Entity.
        em.setJpaVendorAdapter(vendorAdapter);

        Properties jpaProperties = new Properties();
        jpaProperties.setProperty("hibernate.hbm2ddl.auto", "update"); // Tự
động cập nhật schema.
        jpaProperties.setProperty("hibernate.dialect",
"org.hibernate.dialect.H2Dialect"); // Dialect cho H2.
        jpaProperties.setProperty("hibernate.show_sql", "true"); // Hiển thị
SQL trên console.
        jpaProperties.setProperty("hibernate.format_sql", "true"); // Định
dạng SQL đẹp hơn.
        em.setJpaProperties(jpaProperties);
    }
}

```

```

        return em;
    }

    @Bean
    public PlatformTransactionManager transactionManager(EntityManagerFactory
entityManagerFactory) {
        JpaTransactionManager transactionManager = new
JpaTransactionManager();
        transactionManager.setEntityManagerFactory(entityManagerFactory);
        return transactionManager;
    }
}

```

Giải thích:

- **@Configuration** : Đánh dấu lớp này là một lớp cấu hình Spring, chứa các định nghĩa bean.
- **@EnableJpaRepositories(basePackages = "com.example.repository")** : Annotation này kích hoạt tính năng Spring Data JPA repositories. Bạn cần chỉ định **basePackages** nơi Spring sẽ quét để tìm các interface repository của bạn.
- **@EnableTransactionManagement** : Kích hoạt quản lý transaction dựa trên annotation (**@Transactional**).
- **dataSource()** : Định nghĩa một bean **DataSource** để cung cấp kết nối đến cơ sở dữ liệu. Trong ví dụ này, chúng ta sử dụng H2 in-memory database.
- **entityManagerFactory()** : Định nghĩa một bean **LocalContainerEntityManagerFactoryBean** . Đây là thành phần cốt lõi để tích hợp JPA với Spring. Nó chịu trách nhiệm tạo ra **EntityManagerFactory** của JPA. Bạn cần cấu hình:
 - **setDataSource()** : Gán **DataSource** đã định nghĩa.
 - **setPackagesToScan()** : Chỉ định package chứa các Entity classes của bạn để JPA có thể quét và ánh xạ chúng.
 - **setJpaVendorAdapter()** : Cấu hình nhà cung cấp JPA (ví dụ: Hibernate).
 - **setGenerateDdl(true)** cho phép Hibernate tự động tạo bảng dựa trên Entity.

- `setJpaProperties()` : Thiết lập các thuộc tính cụ thể của JPA/Hibernate, như `hibernate.hbm2ddl.auto` (để tự động tạo/cập nhật schema), `hibernate.dialect` , `hibernate.show_sql` , v.v.
- `transactionManager(EntityManagerFactory entityManagerFactory)` : Định nghĩa một bean `PlatformTransactionManager` (cụ thể là `JpaTransactionManager`). Bean này chịu trách nhiệm quản lý các transaction JPA. Nó cần một `EntityManagerFactory` để hoạt động.

1.3. Cấu hình Spring Context

Để Spring có thể tải lớp cấu hình `JpaConfig` , bạn cần cấu hình `AnnotationConfigApplicationContext` trong ứng dụng chính của mình.

Java

```
package com.example;

import com.example.config.JpaConfig;
import com.example.model.User;
import com.example.repository.UserRepository;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Application {

    public static void main(String[] args) {
        ApplicationContext context = new
        AnnotationConfigApplicationContext(JpaConfig.class);

        UserRepository userRepository =
        context.getBean(UserRepository.class);

        // Test the repository
        User user1 = new User();
        user1.setName("Alice");
        user1.setEmail("alice@example.com");
        userRepository.save(user1);

        User user2 = new User();
        user2.setName("Bob");
        user2.setEmail("bob@example.com");
        userRepository.save(user2);
    }
}
```

```
        System.out.println("Users found:");
        userRepository.findAll().forEach(System.out::println);
    }
}
```

2. Cấu hình bằng XML Configuration

Mặc dù JavaConfig là phương pháp được khuyến nghị, bạn vẫn có thể cấu hình Spring Data JPA bằng XML, đặc biệt nếu bạn đang làm việc với một dự án kế thừa.

2.1. Dependencies cần thiết

Các dependency tương tự như JavaConfig, nhưng bạn có thể cần thêm `spring-webmvc` nếu đây là một ứng dụng web MVC truyền thống.

2.2. File cấu hình XML

Tạo một file XML cấu hình Spring (ví dụ: `applicationContext.xml`) và định nghĩa các bean cần thiết.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:jpa="http://www.springframework.org/schema/data/jpa"
       xmlns:tx="http://www.springframework.org/schema/tx"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd
                           http://www.springframework.org/schema/data/jpa
                           http://www.springframework.org/schema/data/jpa/spring-jpa.xsd
                           http://www.springframework.org/schema/tx
                           http://www.springframework.org/schema/tx/spring-
                           tx.xsd">

    <!-- Kích hoạt quét component để tìm các @Service, @Component, v.v. -->
```

```
<context:component-scan base-package="com.example"/>

<!-- Cấu hình DataSource -->
<bean id="dataSource"
class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="org.h2.Driver"/>
    <property name="url" value="jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1"/>
    <property name="username" value="sa"/>
    <property name="password" value=""/>
</bean>

<!-- Cấu hình EntityManagerFactory -->
<bean id="entityManagerFactory"
class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
    <property name="dataSource" ref="dataSource"/>
    <property name="packagesToScan" value="com.example.model"/>
    <property name="jpaVendorAdapter">
        <bean
class="org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter">
            <property name="generateDdl" value="true"/>
        </bean>
    </property>
    <property name="jpaProperties">
        <props>
            <prop key="hibernate.hbm2ddl.auto">update</prop>
            <prop
key="hibernate.dialect">org.hibernate.dialect.H2Dialect</prop>
            <prop key="hibernate.show_sql">true</prop>
            <prop key="hibernate.format_sql">true</prop>
        </props>
    </property>
</bean>

<!-- Cấu hình Transaction Manager -->
<bean id="transactionManager"
class="org.springframework.orm.jpa.JpaTransactionManager">
    <property name="entityManagerFactory" ref="entityManagerFactory"/>
</bean>

<!-- Kích hoạt quản lý transaction dựa trên annotation -->
<tx:annotation-driven transaction-manager="transactionManager"/>

<!-- Kích hoạt Spring Data JPA repositories -->
<jpa:repositories base-package="com.example.repository" entity-manager-
factory-ref="entityManagerFactory" transaction-manager-
ref="transactionManager"/>
```

```
</beans>
```

Giải thích:

- `<context:component-scan>` : Kích hoạt quét component để Spring có thể tìm và đăng ký các bean được đánh dấu bằng `@Component` , `@Service` , `@Repository` , v.v.
- `<bean id="dataSource">` : Định nghĩa bean `DataSource` tương tự như trong JavaConfig.
- `<bean id="entityManagerFactory">` : Định nghĩa bean `LocalContainerEntityManagerFactoryBean` . Các thuộc tính được cấu hình bằng các thẻ `<property>` và `<props>` .
- `<bean id="transactionManager">` : Định nghĩa bean `JpaTransactionManager` .
- `<tx:annotation-driven>` : Kích hoạt quản lý transaction dựa trên annotation (`@Transactional`).
- `<jpa:repositories>` : Kích hoạt Spring Data JPA repositories. Bạn cần chỉ định `base-package` và tham chiếu đến `entityManagerFactory` và `transactionManager` đã định nghĩa.

2.3. Cấu hình Spring Context

Để Spring có thể tải file cấu hình XML, bạn cần cấu hình `ClassPathXmlApplicationContext` trong ứng dụng chính của mình.

Java

```
package com.example;

import com.example.model.User;
import com.example.repository.UserRepository;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class ApplicationXml {

    public static void main(String[] args) {
        ApplicationContext context = new
        ClassPathXmlApplicationContext("applicationContext.xml");
```

```

        UserRepository userRepository =
context.getBean(UserRepository.class);

        // Test the repository
        User user1 = new User();
        user1.setName("Alice");
        user1.setEmail("alice@example.com");
        userRepository.save(user1);

        User user2 = new User();
        user2.setName("Bob");
        user2.setEmail("bob@example.com");
        userRepository.save(user2);

        System.out.println("Users found:");
        userRepository.findAll().forEach(System.out::println);
    }
}

```

3. Cấu trúc dự án mẫu

Để minh họa, đây là cấu trúc dự án cơ bản mà bạn có thể sử dụng:

Plain Text

```

src
├── main
│   ├── java
│   │   ├── com
│   │   │   └── example
│   │   │       ├── Application.java (cho JavaConfig)
│   │   │       ├── ApplicationXml.java (cho XML Config)
│   │   │       ├── config
│   │   │       │   └── JpaConfig.java
│   │   │       ├── model
│   │   │       │   └── User.java
│   │   │       └── repository
│   │   │           └── UserRepository.java
│   └── resources
│       └── applicationContext.xml (cho XML Config)
└── test
    ├── java
    │   ├── com
    │   │   └── example

```

└─ repository
└─ UserRepositoryTest.java (ví dụ test)

com.example.model.User.java

Java

```
package com.example.model;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

```

@Override
public String toString() {
    return "User{" +
        "id=" + id +
        ", name='" + name + '\'' +
        ", email='" + email + '\'' +
        '}';
}
}

```

com.example.repository.UserRepository.java

Java

```

package com.example.repository;

import com.example.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
    // Spring Data JPA sẽ tự động tạo implementation cho interface này.
    // Bạn có thể thêm các phương thức truy vấn tùy chỉnh tại đây.
}

```

4. Test (Ví dụ)

Để kiểm tra cấu hình, bạn có thể viết một test case đơn giản.

Java

```

package com.example.repository;

import com.example.config.JpaConfig;
import com.example.model.User;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.junit.jupiter.SpringExtension;
import org.springframework.transaction.annotation.Transactional;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;

```

```

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = {JpaConfig.class}) // Tải cấu hình
JavaConfig.
@Transactional // Đảm bảo mỗi test chạy trong một transaction riêng và
rollback sau đó.
public class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @Test
    void testSaveUser() {
        User user = new User();
        user.setName("Test User");
        user.setEmail("test@example.com");

        User savedUser = userRepository.save(user);

        assertNotNull(savedUser.getId());
        assertEquals("Test User", savedUser.getName());
    }

    @Test
    void testFindAllUsers() {
        User user1 = new User();
        user1.setName("User A");
        user1.setEmail("usera@example.com");
        userRepository.save(user1);

        User user2 = new User();
        user2.setName("User B");
        user2.setEmail("userb@example.com");
        userRepository.save(user2);

        assertEquals(2, userRepository.findAll().size());
    }
}

```

Giải thích:

- `@ExtendWith(SpringExtension.class)` : Tích hợp JUnit 5 với Spring TestContext Framework.
- `@ContextConfiguration(classes = {JpaConfig.class})` : Chỉ định lớp cấu hình Spring mà test cần tải. Nếu bạn sử dụng XML, bạn sẽ dùng `locations = {"classpath:applicationContext.xml"}` .

- **@Transactional** : Đảm bảo rằng mỗi phương thức test chạy trong một transaction riêng biệt. Theo mặc định, transaction này sẽ được rollback sau khi test hoàn thành, giúp giữ cho cơ sở dữ liệu sạch sẽ giữa các lần chạy test.
- **@Autowired** : Tự động inject `UserRepository` vào test class.

Bằng cách làm theo các bước trên, bạn có thể cấu hình và sử dụng Spring Data JPA trong một dự án Spring truyền thống mà không cần đến Spring Boot.